

Práctica 1: Secuencial vs. Paralelo

Análisis Comparativo del Rendimiento en la Multiplicación de Matrices mediante hilos

Torres Corte Aarón Ulises

Introducción al Cómputo de Alto Rendimiento
Facultad de Ciencias de la Computación, BUAP

Resumen En este reporte se presenta el estudio y la evolución realizada a partir del código base de referencia proporcionado en clase. Se analiza el paso de una transformación matricial por bloques de filas a la implementación del algoritmo de multiplicación de matrices $A_{N \times M} \times B_{M \times Z} = C_{N \times Z}$ con asignación de hilos a nivel de elemento $C_{i,j}$. Se evalúa el cumplimiento de la especificación didáctica de un hilo por casilla para matrices pequeñas y se realiza un análisis de escalabilidad para matrices de gran tamaño, registrando una aceleración empírica máxima de $9,59\times$ con 8 hilos en la máquina de prueba.

Keywords: Hilos · Multiplicación de Matrices · HPC · Granularidad · Speedup · Memory Bandwidth · Ley de Amdahl.

1. Introducción y Fundamentos

El uso de hilos permite introducir los conceptos clave de la programación paralela en arquitectura de memoria compartida. En esta práctica se toma como punto de partida el programa base de referencia presentado en clase, el cual ilustra de forma clara la división de trabajo por bloques de filas, para extenderlo hacia la multiplicación de dos matrices y la evaluación experimental del rendimiento.

1.1. Evolución a partir del Código Base de Referencia

El programa desarrollado en esta práctica mantiene la estructura fundamental y las buenas prácticas introducidas en el código de referencia en C, adaptando la lógica matemática y la granularidad según los requerimientos solicitados:

1. Extensión de la Operación Matemática:

- *Código Base de Referencia:* Presenta la lógica de elevar al cuadrado los elementos de una matriz ($A_{i,j} \leftarrow A_{i,j}^2$) mediante la distribución equitativa del arreglo de filas entre los hilos.

- *Código Implementado (act.2.c)*: Extiende dicho modelo hacia la operación binaria de multiplicación entre dos matrices compatibles $A_{N \times M}$ y $B_{M \times Z}$, verificando la condición de dimensión ($M_1 = M_2$) y calculando cada celda como:

$$C_{i,j} = \sum_{k=0}^{M-1} A_{i,k} \cdot B_{k,j} \quad (1)$$

2. Granularidad y Asignación por Elemento $C_{i,j}$:

- *Código Base de Referencia*: Asigna a cada hilo un rango continuo de filas (`fila_inicio` a `fila_fin`), un esquema muy claro para entender la descomposición de dominio por filas.
- *Código Implementado (act.2.c)*: Implementa la asignación a nivel de elementos individuales de la matriz resultante $C_{i,j}$. Cada casilla lineal $idx \in [0, N \cdot Z - 1]$ se asigna a los hilos y se mapea mediante:

$$i = \lfloor idx/Z \rfloor, \quad j = idx \pmod{Z} \quad (2)$$

Esto permite explorar la premisa didáctica de asignar un hilo por casilla ($p = N \times Z$), así como ajustar p al número de núcleos físicos de la CPU.

3. Incorporación de Algoritmo Secuencial y Métrica Temporal:

- Para realizar la comparación requerida en la práctica, se agregó la función `multiplicar_secuencial()` como línea de base (*baseline*) y la medición de tiempo mediante `clock_gettime(CLOCK_MONOTONIC, ...)` para registrar de forma precisa los tiempos en segundos.

Cuadro 1. Cuadro comparativo entre el código base de referencia y el programa implementado.

Característica	Código Base de Referencia	Código Implementado (act.2.c)
Operación	Elementos al cuadrado ($A_{i,j}^2$)	Multiplicación $A_{N \times M} \times B_{M \times Z} = C_{N \times Z}$
Modelo de División	Bloques continuos de filas	Mapeo de casillas/elementos 1D ($C_{i,j}$)
Evaluación Secuencial	N/A	Función <code>multiplicar_secuencial</code>
Medición de Tiempo	Demostrativo	Resolución con <code>clock_gettime</code>

2. Implementación del Programa

A continuación se presenta el núcleo de la función ejecutada por cada hilo en el programa actual:

```

1 typedef struct {
2     int** matriz_uno;
3     int** matriz_dos;
4     int** matriz_resultante;

```

```

5     int cols_resultante;
6     int casilla_inicio;
7     int casilla_fin;
8     int dim_comp;
9 } DatosHilo;

10
11 void *multiplicar_elemento(void *arg) {
12     DatosHilo* datos = (DatosHilo*)arg;
13     for(int idx = datos->casilla_inicio; idx < datos->
14         casilla_fin; idx++) {
15         int fila = idx / datos->cols_resultante;
16         int col = idx % datos->cols_resultante;
17         int suma = 0;
18         for (int k = 0; k < datos->dim_comp; k++) {
19             suma += datos->matriz_uno[fila][k] * datos->
20             matriz_dos[k][col];
21         }
22         datos->matriz_resultante[fila][col] = suma;
23     }
24     return NULL;
25 }

```

Listing 1.1. Función de multiplicación por casilla en el código implementado.

3. Resultados Experimentales y Evaluación de Rendimiento

Se realizaron pruebas empíricas ejecutando el binario compilado (-O2) de forma nativa en el sistema local, evaluando el comportamiento bajo distintas dimensiones de matriz y configuraciones de hilos.

3.1. 1. Enfoque Didáctico Estricto: Un Hilo por Elemento ($p = N \times Z$)

Siguiendo la indicación de la práctica de asignar un hilo por cada elemento $C_{i,j}$, en matrices de tamaño didáctico (como 3×3 o 5×5), el programa crea exitosamente 9 o 25 hilos, asignando exactamente una casilla a cada uno.

Sin embargo, al aplicar este enfoque en una matriz de 500×500 (250,000 hilos), la sobrecarga de llamadas al sistema y cambios de contexto degrada el tiempo a 6,028 s frente a 0,517 s secuencial ($S = 0,10\times$).

3.2. 2. Enfoque de Escalamiento Ajustado a CPU ($p = 8$ Hilos)

Ajustando el número de hilos al total de núcleos de la CPU ($p = 8$), se registraron los tiempos reales de ejecución presentados en la Tabla 2.

Cuadro 2. Resultados experimentales de tiempos de ejecución y aceleración (*Speedup*) medidos de forma nativa en la máquina de prueba ($p = 8$ hilos).

Dimensiones ($N \times M \times Z$)	Hilos	$T_{\text{secuencial}}$ (s)	T_{paralelo} (s)	Speedup (S)	Resultado
$5 \times 5 \times 5$	8	0.000001	0.000214	$0,00\times$	Secuencial más rápido
$20 \times 20 \times 20$	8	0.000033	0.000211	$0,16\times$	Secuencial más rápido
$100 \times 100 \times 100$	8	0.004185	0.000789	$5,30\times$	Inflexión Empírica ($S > 1$)
$300 \times 300 \times 300$	8	0.111311	0.016796	$6,63\times$	Aceleración Paralela
$500 \times 500 \times 500$	8	0.517333	0.069659	$7,43\times$	Aceleración Paralela
$800 \times 800 \times 800$	8	2.190288	0.240797	$9,10\times$	Aceleración Paralela
$1000 \times 1000 \times 1000$	8	4.430246	0.462081	$9,59\times$	Aceleración Máxima
$1200 \times 1200 \times 1200$	8	7.529368	0.847012	$8,89\times$	Saturación Ligera de Bus

4. Análisis de Arquitectura de Computadoras y Cómputo de Alto Rendimiento

Al analizar los datos nativos medidos en la máquina local (Tabla 2), destacan tres aspectos fundamentales del rendimiento:

4.1. 1. Escalamiento Excepcional en Matrices Gigantes

En las pruebas nativas sobre 1000×1000 , la versión secuencial requirió **4.430 segundos**, mientras que la versión paralela resolvió la multiplicación en tan solo **0.462 segundos**, alcanzando el pico de aceleración máxima de **$9.59\times$** . En 1200×1200 , la ejecución secuencial tomó **7.529 segundos**, reduciéndose a **0.847 segundos** en paralelo ($S = 8,89\times$).

4.2. 2. Inicio de la Saturación de Bus de Memoria en 1200×1200

Al comparar el *speedup* entre 1000×1000 ($9,59\times$) y 1200×1200 ($8,89\times$), se evidencia de forma empírica la primera inflexión descendente por saturación del bus de memoria RAM (*Memory Bandwidth Bottleneck*). Al superar los 17,28 MB de memoria activa requeridos por las tres matrices, la huella desborda la memoria caché L3, obligando a los 8 hilos a competir por la lectura/escritura en RAM.

4.3. 3. Dependencia del Punto de Inflexión respecto al Hardware

Es fundamental señalar que los valores exactos de tiempo y el punto de inflexión donde $S > 1$ **no constituyen una constante universal**, sino un resultado empírico propio de la arquitectura física utilizada. Varían según:

- El número de núcleos físicos y lógicos de la CPU.
- La frecuencia de reloj y la capacidad de las memorias caché L1, L2 y L3.
- La velocidad y ancho de banda de la memoria RAM principal.
- Los flags de optimización utilizados en la compilación (-O2).

5. Conclusión

Las mediciones nativas en la máquina de prueba confirman el poder del cómputo paralelo mediante hilos en arquitecturas multinúcleo. Para problemas pequeños (5×5 y 20×20), el algoritmo secuencial domina al evitar la sobrecarga de administración del kernel. No obstante, para matrices gigantes de 1200×1200 , la versión paralela redujo el tiempo de procesamiento de 7,52 s a apenas 0,84 s ($8,89\times$ más rápido), demostrando la eficiencia del modelo implementado para acelerar cargas de alta densidad computacional.